

AIMLCZG557 Practice Questions

Combined print packet for Artificial and Computational Intelligence

Generated locally on 2026-06-18

Contents

Section	Source	Pages
EC2 Sample 1	EC2_Sample1.pdf	2
EC2 Sample 2	EC2_Sample2.pdf	3
Assignment 1 - Problem Set 7	course-assets/assignments/Assignment 1 - Problem Set 7.pdf	7
Previous-year Sample Questions	Webinar 02 - Sample Questions from Previous Year Exam.pptx	32
Hill Climbing Travel Optimization	Webinar 02 - Hill Climbing Travel Optimization.pptx	5
Topic-wise Collected Questions	practice/sample_questions_topic_wise.md	2
Sprint Mock 01	practice/mocks/sprint-mock-01.md	3

Note: instructor/source artifacts are bundled here for local printing only. The hill-climbing deck is included as extracted slide text because LibreOffice produced an empty PDF for that source file.

EC2 Sample 1

EC2_Sample1.pdf

Local EC2 sample question paper.

Birla Institute of Technology & Science, Pilani
Work Integrated Learning Programmes Division
First -Semester 2025-2026

Mid-Semester Test
(EC-2)

Course No. : AIMLZG557
Course Title : Artificial and Computational Intelligence
Nature of Exam : Closed Book
Weightage : 30%
Duration : 2 Hours
Date of Exam : XX/XX/XXXX (AN)

No. of Pages	= 2
No. of Questions	= 4

Note to Students:

1. Please follow all the *Instructions to Candidates* given on the cover page of the answer book.
2. All parts of a question should be answered consecutively. Each answer should start from a fresh page.
3. Assumptions made if any, should be stated clearly at the beginning of your answer.

- Q.1 a) Identify and explain two application domains of AI and discuss the challenges AI faces in these domains.
- b) Differentiate between tree search and graph search. Provide an example scenario where graph search is more suitable.
- c) Compare and contrast PSO and ACO as population-based optimization algorithms.
- d) Define the transition probability in Ant Colony Optimization (ACO) and explain the role of parameters α (pheromone importance) and β (heuristic importance).

[2+2+2+2=8 Marks]

- Q.2 A **warehouse robot** begins at C1, tasked with collecting Items (It) and reaching the Dock (DV) at C4 on a 4x4 grid. The robot must navigate hazards like blocked cells (#), Spills (SP) with high movement costs, while leveraging Energy packs (E) to reduce costs. Using the A* algorithm, the robot calculates the optimal path by considering both actual path cost and a heuristic. The agent is equipped with four distinct actions: Move Up, Move Down, Move Left, and Move Right. However, it cannot move into blocked cells. The warehouse is represented as a 4×4 grid below:

	1	2	3	4
A	It	E	#	SP
B	SP	It	E	SP
C	S	E	It	DV
D	It	SP	SP	E

Entering **SP**: +12; collecting **It**: -8 (reward); energy pack E: +1.

Heuristic. $h(n)$ = Manhattan(robot, DV) + (# remaining Items) + (# adjacent SP to robot in resultant state).

- a) Expand and depict the search tree for exactly 2 levels. (Level 0 = Start).
- b) Calculate Path costs and heuristics (g/h/f) for all generated nodes.
- c) Apply IDA* search and show the first 3 CLOSED list updates with OPEN/CLOSED status.

[3+3+3 = 9 Marks]

- Q.3 A large electronics manufacturing facility wants to ensure that only defect-free components are shipped to customers. An **Automated Quality Control System** is being designed with AI-enabled sensors and actuators. The system inspects each component on the conveyor belt, identifies defects by comparing them with a database of standards, and classifies them into either **Accept (Pass)** or **Reject (Fail)** categories.

System Details:

- Sensors are installed at critical checkpoints along the production line to detect cracks, misalignments, missing solder points, and irregular voltages.
- Defective components are automatically diverted to the **Reject bin** using conveyor diverters, while non-defective ones proceed to **Packaging**.
- The system must also maintain logs and raise alerts if defect rates exceed a threshold.

- a. Provide the complete problem formulation.
 - b. Provide the PEAS description.
 - c. Identify the various dimensions of the task environment with appropriate justification for each in no more than 30 words.
- [2+2+2=6 Marks]**

- Q.4 A hospital must create a **weekly schedule** for 5 nurses (N1–N5) across **three shifts per day** (Morning, Evening, Night). The schedule must ensure that all shifts are covered while preventing overwork. A nurse cannot be assigned **more than 5 shifts per week (Mon-Fri)**.

You are tasked with designing a **Genetic Algorithm** to generate feasible schedules. Each chromosome encodes the assignment of shifts to nurses as a binary string of length 15 (5 nurses × 3 shifts). The fitness function rewards full coverage and penalizes uncovered shifts and overworked nurses.

- a) Represent 4 random chromosome states with their schedules and calculate the **fitness score** for each.
- b) Describe the **Selection, Crossover, and Mutation** process for this problem. Demonstrate one full iteration using a tournament selection of 2, a 1-point crossover, and a single bit-flip mutation. Clearly show the generated offspring and their fitness.

[3+4=7 Marks]

EC2 Sample 2

EC2_Sample2.pdf

Local EC2 makeup/sample paper source.

Birla Institute of Technology & Science, Pilani
Work-Integrated Learning Programmes Division
Second Semester 20XX-XX
Mid Semester Examination(Makeup)

Course No. : ZG557
Course Title : Artificial & Computational Intelligence
Nature of Exam : Closed Book
Weightage : 30%
Duration : 1.5 Hours
Date of Exam : XX/ XX / 2020, AN
PM

No. of Pages	= 4
No. of Questions	= 4

Time of Exam: 2:00 PM to 3:30

Note:

1. Please follow all the *Instructions to Candidates* given on the cover page of the answer book.
2. All parts of a question should be answered consecutively. Each answer should start from a fresh page.
3. **Questions must be answered only in the page numbers mentioned against each question. In case you require more pages to answer a question, you can continue from page #21**
4. Assumptions made if any, should be stated clearly at the beginning of your answer.

Answer all the questions

Question -1 [1.5 + 1.5 + 1.5 + 2 +1.5 = 8 M]

[Page 03 - 09]

(1-a) Provide one point for and against Turing Test to determine if a software piece is intelligent

For questions (1-b) and (1-c), answer if forward search (data driven search) or backward search (goal driven search) will be more appropriate with one good reason for the scenarios given:

(1-b) You have met a person Dr. Rajiv who claims to be your distant cousin. You need to verify his claim by finding some common ancestor who is a few generations older.

(1-c) A system that will help human to classify plants by species

For questions (1-d) and (1-e) consider the following scenario:

A farmer with his wolf, goat, and cabbage come to the edge of a river they wish to cross. There is a boat at the river's edge, but, of course, only the farmer can row. The boat also can carry only two things (including the rower) at a time. If the wolf is ever left alone with the goat, the wolf will eat the goat; similarly, if the goat is left alone with the cabbage, the goat will eat the cabbage. Devise a sequence of crossings of the river so that all four characters arrive safely on the other side of the river.

(1-d) Devise a concise representation of a state (considering all possible uses of this in search algorithms taught to you). As per the representation devised provide write down the representation of a start state and the goal state.

Provide the data structure description of a state. Specify the fields (with details of what goes into), pointers to be kept in the node.

Answers both in pseudo code, pictorial rep, in description are all allowed. [1.5 M]

State diagram, solution to the problem are not expected here

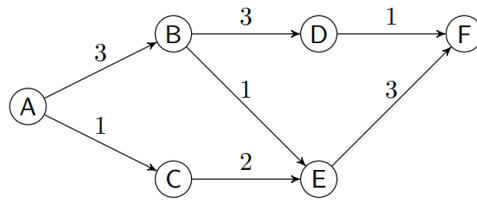
(1-e) Which of DFS or BFS is most preferred for this problem? Why.

DFS is recommended. This will lead to the least number of nodes being expanded. This will become obvious when the state diagram is drawn.

Question -2 [4 + 2 + 1 = 7 M]

[Page 10 - 15]

(2-a) Consider the following state space and the heuristic function associated. The initial state is A and the goal state is F.



Show the result of Uniform cost search algorithm on this search tree.

(2-b) Is the uniform cost search (as presented in class and Text Book) a tree search or a graph search? Explain why should it be this way?

(2-c) When the heuristic function $h(n) \geq \epsilon$ (where ϵ is a small positive constant) the states expanded by A* algorithm will be a subset of uniform cost search. Why or why not?

Question -3 [3+2+ 3 +2 + 1 = 11 M]

[Page 16 - 22]

The *sliding-tile puzzle* consists of three black tiles, three white tiles, and an empty space in some order. The goal is to arrive at the goal configuration shown below by legal moves [B- Black tile, W-White tile].

B	B	B		W	W	W
---	---	---	--	---	---	---

The puzzle has two legal moves with associated costs

Move #1: A tile may move into an adjacent empty location. This has a cost of 1.

Move #2: A tile can hopover one or two other tiles into the empty position. This has a cost equal to the number of tiles jumped over

(2-a) What are measures using which the complexity of a state space can be measured. List each of them and provide the corresponding expression / value for this problem.

Listing of space, branching , - 1Mark, Corresponding values for this problem - 2M

(2-b) Write down an admissible heuristic function $h(n)$ for this problem with a formal description. Explain why this proposed heuristic is admissible.

1Mark for heuristic function and 1Mark for explaining

$h(n)$ = # of white tiles on the wrong side of each black tile

- Breadth-first search is admissible, monotonic and very uninformed.
- Counting tiles out of the goal position is a simple useful heuristic that is admissible.
- A form of this heuristic would be to count all white tiles on the wrong side of each black tile.
- For example for the following configuration, there are 3 Ws on the right side of 1st B, and 2 Ws on the right side of both the 2nd and 3rd Bs.
- Therefore $h(n) = 3+2+2 = 7$ B_WBBWW
- Counting tiles out of place is not monotonic, but is more informed than breadth-first search. An easy way to prove that a heuristic is non-monotonic is to watch its value change as it goes along a search path.

(2-c) Generate the A* search tree (using tree search) starting with the following start state for depth up to 3 or until the goal state is reached, whichever is earlier.

W	B	W	B	B	W	
---	---	---	---	---	---	--

3Marks for : Annotate each node and edges with appropriate values $f(n)$, $h(n)$ and $g(n)$ for better understanding. A search tree without such annotations will not get full credit here.

(2-d) Argue formally the optimality of the solution obtained.

Valid argument - 2M, Good attempt in right direction - 1M

(2-e) Will the solution obtained be different if the heuristic is also consistent in addition to being admissible. Why or why not?

0.5 , 0.5 for reasoning.

Question -4 [4M]

[Page 22 - end]

You have been asked to solve below linear equation problem with multiple variables:

$$2a + 7b - 5c + d = 0$$

where a, b,c, and d are integers in the range [-20,20] (equation can have more than one solution)

You have decided to solve this question using genetic algorithm. Show all the steps involved in solving this using Genetic Algorithm.

Number each of your steps, and provide appropriate title to those steps. (No need to get the final solution, just the first iteration of each step involved in the algorithm)

Most likely representation of chromosome could be a list with 4 integers with each integer representing variables a,b,c or d. For example [3,2,3,1] would imply a=3,b=2,c=3,d=1.

Generate random population (list of lists)

Fitness function - use a fitness function (most likely $f(x) = 2a + 7b - 5c + d$) and calculate the fitness of each individual

Apply some selection criteria based on fitness value (could be roulette or the best fit etc.) to come up with initial population

Apply crossover and mutation (how students chose to incorporate crossover and mutation may differ but it should be consistent)

Assignment 1 - Problem Set 7

[course-assets/assignments/Assignment 1 - Problem Set 7.pdf](#)

Emergency route-planning practice using Greedy Best First Search.



Birla Institute of Technology & Science, Pilani

Work-Integrated Learning Programmes Division

MTech in Artificial Intelligence and Machine Learning

S2_2025-2026, AIMLCZG557/AECLZG557

Assignment 1 – PS7 - Emergency Plan - [Weightage 12%]

Read through this entire document very carefully before you start!

Design a Problem Solving Agent for Emergency Route Planning using Greedy Best First Search

Problem Statement

During heavy rainfall in Chennai, several roads are flooded and rescue teams must quickly reach affected areas. A disaster management control center wants to develop a Problem Solving Agent that can identify a path from a Start Location to a Goal Location using the Greedy Best First Search (GBFS) algorithm.

The agent should always expand the node that appears closest to the goal based on the heuristic value $h(n)$.

Input

The program should accept:

- A graph with nodes and edges
- Heuristic values for every node
- Start node
- Goal node

Output

The program should display:

- Order of node expansion
- Final path from source to goal

- Total cost/path length (optional)

Algorithm Requirement

Use the following strategy:

$$f(n) = h(n)$$

Where:

- $h(n)$ = Estimated distance from current node to goal.

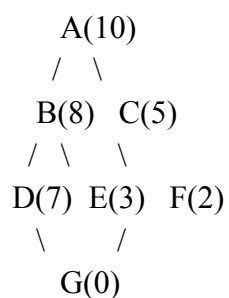
The node with the smallest heuristic value should be expanded first.

Assignment Tasks

1. Represent the city map as a graph.
2. Implement Greedy Best First Search in Python.
3. Use heuristic values to guide the search.
4. Print:
 - o Visited nodes
 - o Optimal path
 - o Total cost
5. Compute:
 - o Time Complexity
 - o Space Complexity

Test Case 1

Graph



Goal Node = G

Adjacency List

Node	Connected To
A	B, C
B	D, E
C	F
D	G
E	G
F	G

Heuristic Values

Node	h(n)
A	10
B	8
C	5
D	7
E	3
F	2
G	0

Input

Start Node : A

Goal Node : G

Expected Expansion Order

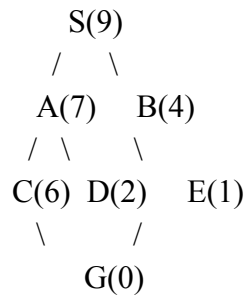
$A \rightarrow C \rightarrow F \rightarrow G$

Expected Output Path

$A \rightarrow C \rightarrow F \rightarrow G$

Test Case 2

Graph



Goal Node = G

Adjacency List

Node Connected To

S	A, B
A	C, D
B	E
C	G
D	G
E	G

Heuristic Values

Node	h(n)
S	9
A	7
B	4
C	6
D	2
E	1
G	0

Input

Start Node : S

Goal Node : G

Expected Expansion Order

$S \rightarrow B \rightarrow E \rightarrow G$

Expected Output Path

$S \rightarrow B \rightarrow E \rightarrow G$

1. Explain the PEAS (Performance Measure, Environment, Actuators, Sensors) for the routing agent.
2. Explain the environmental characteristics for the given problem.
3. Explain the heuristic function used for the problem
4. Use appropriate data structures and implement the Greedy Best First Search algorithm.
5. The starting point is to be obtained from the user as input.
6. Print the space and time complexity as part of your implementation. Do not calculate theoretically.

Deliverables:

- PDF document **designPSXX_<group id>.pdf** detailing your solution.
- **[Group id] _Contribution.xlsx** mentioning the contribution of each student in terms of percentage of work done. Columns must be “Student Registration Number”, “Name”, “Percentage of contribution out of 100%”. If a student did not contribute at all, it will be 0%, if all contributed then 100% for all.
- **inputPSXX.txt** file used for testing
- **outputPSXX.txt** file generated while testing
- .py file containing the python code. Create a single notebook. Do not fragment your code into multiple files
- Zip all of the above files including the design document in a folder with the name:
 - **[Group id] _A1_PSXX_XXXXXXXXXX.zip** and submit the zipped file.
 - Group Id should be given as **Gxxx** where **xxx** is your group number. For example, if your group is 26, then you will enter **G026** as your group id.

Instructions:

- It is compulsory to make use of the data structure(s) / algorithms mentioned in the problem statement.

- Ensure that all data structure insert and delete operations throw appropriate messages when their capacity is empty or full. Also ensure basic error handling is implemented.
- For the purposes of testing, you may implement some functions to print the data structures or other test data. But all such functions must be commented before submission.
- Make sure that you read, understand, and follow all the instructions
- Ensure that the input, prompt and output file guidelines are adhered to. Deviations from the mentioned formats will not be entertained.
- The input, prompt and output samples shown here are only a representation of the syntax to be used. Actual files used to evaluate the submissions will be different. Hence, do not hard code any values into the code.
- Please note that the design document must include:
 - One alternate way of modeling the problem with the performance implications.
 - Writing a good technical report and well documented code is an art. Your report cannot exceed 4 pages. Your code must be modular and quite well documented.
 - You may ask queries in [this dedicated discussion section](#). Beware that only hints will be provided and queries asked **in other channels like MS Teams, emails, Taxila inbox will not be responded to.**

Deadline

1. The strict deadline for submission of the assignment is **8th June 2026 11:55 PM IST (Monday)**.
2. The deadline has been set considering extra days from the regular duration in order to accommodate any challenges you might face. No further extensions will be entertained.
3. Late submissions will be evaluated as below. Submissions made between:
 - a. 8th June 2026 11:56PM to 9th June 2026 11:55PM → Deduct 2M for late submission and evaluation for 10 marks.
 - b. 9th June 2026 11:56PM to 10th June 2026 11:55PM → Deduct 6M for late submission and evaluation for 6marks.
 - c. Beyond 10th June 2026 11:55PM → No evaluations, 0 marks will be awarded.

How to submit

- This is a group assignment.
- Each group has to make one submission (only one, no resubmission) of solutions.
- Each group should zip all the deliverables in one zip file and name the zipped file as [Group id] _A1_PSXX_XXXXXXXXXX.zip and submit the zipped file.
- Group Id should be given as Gxxx where xxx is your group number. For example, if your group is 26, then you will enter G026 as your group id.
- Assignments should be submitted via Taxila > Assignment section. Assignments

submitted via other means like email etc. will not be graded.

Evaluation

- The assignment carries **12 Marks**.
- Grading will depend on:
- Fully executable code with all functionality working as expected
 - Well-structured and commented code
 - Accuracy of the design document.
 - Every bug in the functionality will have negative marking.
 - Marks will be deducted if your program fails to read the input file used for evaluation due to change / deviation from the required syntax.
- We encourage students to take the upcoming assignments and examinations seriously and submit only original work. Please note that plagiarism in assignments will be taken seriously. Refer to the detailed policy [here](#).
- Source code files which contain compilation errors will get at most 25% of the value of that question.

Readings

Artificial Intelligence: A Modern Approach Textbook by Peter Norvig and Stuart J. Russell, 4th Edition.

Previous-year Sample Questions

Webinar 02 - Sample Questions from Previous Year Exam.pptx

Converted from the local PPTX source for printing.



Sample questions from previous year exam

BITS Pilani
Pilani Campus



Question 1:

A Rescue Agent is deployed in a flood-affected city to deliver medicines from the base camp at (X1, Y1) to a relief shelter located at (X5, Y4). The agent must efficiently plan the route to ensure the timely delivery of the medical supplies. The city is represented as a 5x5 grid. The agent is equipped with four distinct actions: Move Up, Move Down, Move Left, and Move Right. However, it cannot move into waterlogged and blocked cells. The city is represented as a 5x5 grid below:

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Legend:

S = Start (Base Camp)

G = Goal (Relief Shelter)

N = Normal cell (cost +1)

W = Waterlogged (+5 cost)

E = Energy pack (−2 cost)

B = Blocked (not traversable)

Heuristic:

$H(n) = \text{Manhattan Distance to Goal} +$
No. of Waterlogged cells adjacent to position

For the above problem do the following:

- a) Expand and depict the search tree for exactly 3 levels. (Level 0 = Start).
- b) Calculate path cost and heuristic values for all nodes at Level 1–3.
- c) Apply A* search and show all steps.

Manhattan distance = For two points $(P(x_1, y_1))$ and $(Q(x_2, y_2))$, the formula is:

$$D = |x_1 - x_2| + |y_1 - y_2|$$

Initial assumptions from given statement



Grid rules used

Traversable cells: S, N, E, G

Non-traversable cells: W, B

Costs:

Heuristic:

$H(n)$ = Manhattan Distance to Goal +

No. of Waterlogged cells adjacent to position

Goal = (X5, Y4)

Cell	Cost
N	+1
E	-2
G	+1
W	Not allowed
B	Not allowed

Solution - a) Expand and depict the search tree for exactly 3 levels. (Level 0 = Start).



Level 0 = Start

Level 1 = nodes one move from Start

Level 2 = nodes two moves from Start

Level 3 = nodes three moves from Start

Level 0:

(X1,Y1) S

5 minutes for expanding the search tree up to level 3

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Solution - a) Expand and depict the search tree for exactly 3 levels. (Level 0 = Start).



Level 1:

From (X1,Y1):

├── Down → (X2,Y1) N
└── Right → (X1,Y2) N

Level 2:

From (X2,Y1):

└── Down → (X3,Y1) N

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Solution - a) Expand and depict the search tree for exactly 3 levels. (Level 0 = Start).



From (X1,Y2):

No valid new child

Right → (X1,Y3) W, not allowed

Down → (X2,Y2) W, not allowed

Left → (X1,Y1), already visited

Level 3:

From (X3,Y1):

└ Right → (X3,Y2) N

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Let's calculate the path costs and heuristic values for levels 1 to 3

Heuristic $h(n)$ = Manhattan Distance to Goal +

No. of Waterlogged cells adjacent to position

Evaluation function $f(n) = g(n) + h(n)$

$g(n)$ = The **actual path cost**. This represents the exact cost to reach the current node n from the starting node.

$f(n)$: The **total estimated cost**. This is the projected cost of the entire cheapest solution that passes through node n

5 minutes for calculating the values: $g(n)$, $h(n)$ and $f(n)$

Solution - b) Calculate path cost and heuristic values for all nodes at Level 1–3.



B) Path cost and heuristic values for Levels 1 to 3

Level 1

Node	Path	$g(n)$	Manhattan distance	Adjacent W cells	$h(n)$	$f(n)$
(X2,Y1)	$S \rightarrow X2Y1$	1	6	1	7	8
(X1,Y2)	$S \rightarrow X1Y2$	1	6	2	8	9

$H(n) = \text{Manhattan Distance to Goal} +$

No. of Waterlogged cells adjacent to position

Manhattan distance (X2Y1) to goal(X5Y4): $|5 - 2| + |4 - 1| = 3 + 3 = 6$

Manhattan distance (X1Y2) to goal(X5Y4): $|5 - 1| + |4 - 2| = 4 + 2 = 6$

$f(n) = g(n) + h(n)$

Where:

$g(n)$ = path cost

$h(n)$ = heuristic

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Solution - b) Calculate path cost and heuristic values for all nodes at Level 1–3.



Level 2

Node	Path	$g(n)$	Manhattan distance	Adjacent W cells	$h(n)$	$f(n)$
(X3,Y2)	$S \rightarrow X2Y1 \rightarrow$ $X3Y1 \rightarrow$ $X3Y2$	3	4	2	6	9

Level 2

For **(X3,Y2)**, the adjacent waterlogged cells are:

$(X2,Y2) = W$

$(X3,Y3) = W$

So:

$h(X3,Y2) = 4 + 2 = 6$

$f(n) = g(n) + h(n) = 3 + 6 = 9$

Solution - b) Calculate path cost and heuristic values for all nodes at Level 1–3.



Level 3

Node	Path	$g(n)$	Manhattan distance	Adjacent W cells	$h(n)$	$f(n)$
(X3,Y1)	$S \rightarrow X2Y1 \rightarrow X3Y1$	2	5	0	5	7

5 minutes for A* algorithm :

Find the valid child, add to open list, choose the next node

Solution - C) A*



Step 0: Start

Expand (X1,Y1).

Node	g(n)	h(n)	f(n)
(X1,Y1)	0	7	7

Step 1: Expand (X1,Y1)

Valid children:

Child	Cell	g(n)	h(n)	f(n)
(X2,Y1)	N	1	7	8
(X1,Y2)	N	1	8	9

Choose lowest f:(X2,Y1)

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Solution - C) A*



Step 2: Expand (X2,Y1)

Valid child:

Node	$g(n)$	$h(n)$	$f(n)$
(X3,Y1)	2	5	7
(X1,Y1)	1	0	7

(X1,Y1) already visited. Therefore
Open list:

Child	Cell	$g(n)$	$h(n)$	$f(n)$
(X3,Y1)	N	2	5	7

Choose:(X3,Y1)

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Solution - C) A*



Step 3: Expand (X3,Y1)

Valid child:

Child	Cell	$g(n)$	$h(n)$	$f(n)$
(X3,Y2)	N	3	6	9

Open list:

Node	$g(n)$	$h(n)$	$f(n)$
(X3,Y1)	2	5	7
(X3,Y2)	3	6	9

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Tie. Choose **(X3,Y2)** because it is closer to the goal, that is, it has lower heuristic value.

Solution - C) A*



Step 4: Expand (X3,Y2)

Valid children:

Child	Cell	g(n)	h(n)	f(n)
(X4,Y2)	N	4	3	7

Choose:(X4,Y2)

Step 5: Expand (X4,Y2)

Valid children:

Child	Cell	g(n)	h(n)	f(n)
(X5,Y2)	N	5	2	7
(X4,Y3)	N	5	4	9

Choose: (X5,Y2)

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Solution - C) A*



Step 6: Expand (X5,Y2)

Valid children:

Child	Cell	g(n)	h(n)	f(n)
(X4,Y2)	N	4	3	7
(X5,Y3)	E	3	1	4

Since **(X5,Y3)** is an energy cell:

$$g(X5,Y3)=5+(-2)=3$$

Choose:

(X5,Y3)

	Y1	Y2	Y3	Y4	Y5
X1	S	N	W	N	N
X2	N	W	N	N	B
X3	N	N	W	E	N
X4	B	N	N	W	N
X5	N	N	E	G	N

Solution - C) A*



Step 7: Expand (X5,Y3)

Valid children:

Child	Cell	$g(n)$	$h(n)$	$f(n)$
(X5,Y4)	G	4	1	5
(X4,Y3)	N	4	4	8

Choose:

(X5,Y4)

Goal reached.

Solution - C) A*



Final path cost

Step	Cell	Cost
Start	(X1,Y1) S	0
Down	(X2,Y1) N	+1
Down	(X3,Y1) N	+1
Right	(X3,Y2) N	+1
Down	(X4,Y2) N	+1
Down	(X5,Y2) N	+1
Right	(X5,Y3) E	-2
Right	(X5,Y4) G	+1

Total cost=0+1+1+1+1+1-2+1=4

Therefore, the path is:

(X1,Y1)→(X2,Y1)→(X3,Y1)→(X3,Y2)→(X4,Y2)→(X5,Y2)→(X5,Y3)→(X5,Y4)

PEAS and Environment example



Question 2:

A large supermarket chain wants to implement an Autonomous Checkout System where customers can pick items and walk out without scanning them manually. The system uses cameras, weight sensors, and RFID scanners to track selected items, update the bill, and charge customers automatically through a mobile app.

- Provide the complete problem formulation.
- Provide the PEAS description.
- Identify the various dimensions of the task environment with appropriate justification for each.

Problem formulation, PEAS and Environment example



A supermarket wants an intelligent system that can automatically identify the items picked by customers, update their shopping bill in real time, and charge them when they leave the store, without manual barcode scanning or cashier involvement.

Initial State

The customer enters the supermarket and authenticates through the mobile app.

Problem formulation



Initial conditions:

Customer is identified

Shopping cart/basket is empty

Bill amount = 0

Customer location = store entrance

All store shelves have known item positions and inventory counts

Payment method is linked to the customer account

Problem formulation



Goal State

The customer exits the store, and the system has:

- Correctly identified all items taken by the customer
- Removed items that were returned to shelves
- Generated the final bill
- Charged the customer automatically
- Updated store inventory
- Sent receipt to the customer

Problem formulation



State Space

A state represents the current situation of the supermarket checkout process. It may include:

Customer ID

Customer location inside the store

Items currently picked by the customer

Items returned by the customer

Shelf inventory status

Sensor readings from cameras, RFID scanners, and weight sensors

Current bill amount

Payment status

Exit status

Problem formulation



Example state:

Customer C101 is near Shelf A.

Customer has picked Milk and Bread.

Milk shelf weight decreased by one unit.

RFID tag for Bread detected in customer basket.

Current bill = ₹120.

Problem formulation



Actions

The system and customer actions include:

Customer enters store

- Customer picks an item

- Customer returns an item

- Customer moves between aisles

- System detects item using cameras, sensors, or RFID

- System updates cart

- System recalculates bill

- Customer exits store

- System verifies cart

- System charges customer

- System updates inventory

- System sends receipt

- System raises alert if uncertainty or fraud is detected

Problem formulation



Transition Model

The transition model describes how the system moves from one state to another after an action.

Examples:

If customer picks an item:

Shelf weight decreases

Camera detects hand-item interaction

RFID scanner detects item movement

Customer cart is updated

Bill is increased

Problem formulation



If customer returns an item:

Shelf weight increases

Camera detects item placement

RFID tag returns to shelf region

Customer cart is updated

Bill is reduced

If customer exits:

System finalizes detected items

Payment is processed

Receipt is generated

Inventory is updated

Problem formulation



Goal Test

The goal test checks whether checkout is complete.

Customer has exited the store

All selected items are correctly identified

Final bill is accurate

Payment is successful

Receipt is generated

Inventory is updated

Problem formulation



Path Cost

The path cost may include multiple factors:

Incorrect item detection cost

Delay in billing cos

Customer waiting time

Payment failure cost

Inventory mismatch cost

False fraud alert cost

Sensor processing cost

Customer dissatisfaction cost

The system should minimize:

Billing errors + checkout delay + sensor uncertainty + fraud risk + customer inconvenience

PEAS Component	Description
Performance Measure	Accurate item detection, correct billing, fast checkout, low customer waiting time, reduced theft, low false alerts, successful payment, accurate inventory update, high customer satisfaction
Environment	Supermarket aisles, shelves, products, customers, shopping carts/baskets, entry gate, exit gate, mobile app, payment gateway, store inventory database
Actuators	Mobile app notifications, digital bill update, automatic payment trigger, receipt generation, inventory update system, exit gate control, alert system for staff/security
Sensors	Cameras, weight sensors on shelves, RFID scanners, entry/exit sensors, mobile app authentication, payment confirmation system, inventory database inputs

Environment



Dimension	Classification	Justification
Observability	Partially observable	Sensors may miss or misread actions
Agents	Multi-agent	Many customers, staff, and systems interact
Determinism	Stochastic	Sensor errors and human behavior introduce uncertainty
Episodic/Sequential	Sequential	Earlier detections affect final billing
Static/Dynamic	Dynamic	Customers and items change continuously
Discrete/Continuous	Hybrid, mostly continuous	Item identities are discrete, but tracking and sensing are continuous
Known/Unknown	Mostly known	Store rules are known, but behavior and sensor readings are uncertain

Hill Climbing Travel Optimization

Webinar 02 - Hill Climbing Travel Optimization.pptx

Slide text extraction of the hill-climbing lab/practice deck.

Hill Climbing Travel Optimization

Extracted from local PPTX source.

Slide 1

ACI - Lab Session 2

Slide 2

Optimizing attributes for fuel economy Hill Climbing Algorithm Autonomous car driving agent

Slide 3

Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing value to find the peak of the mountain or best solution to the problem.

It terminates when it reaches a peak value where no neighbor has a higher value.

One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.

Hill Climbing Algorithm

Slide 4

It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.

A node of hill climbing algorithm has two components which are state and value .

Hill Climbing is mostly used when a good heuristic is available.

In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

Hill Climbing Algorithm

Slide 5

function HILL-CLIMBING(problem) returns a state that is a local maximum
current ← MAKE-NODE(problem.INITIAL

loop do neighbor ← a highest-valued successor of current if neighbor.VALUE ≤ current.VALUE
then return current.STATE
current ← neighbor
Hill climbing - function

Slide 6

Performance measure of an automated driver Desirable qualities include getting to the correct destination;

minimizing fuel consumption

and wear and tear; minimizing the trip time or cost; minimizing violations of traffic laws and disturbances to other drivers; maximizing safety and passenger comfort; maximizing profits. Obviously, some of these

goals conflict, so tradeoffs will be required.

Fuel economy is calculated by using the formula $\text{Average Fuel Consumption (Km/L)} = \frac{\text{Distance travelled}}{\text{Litres}}$

of fuel consumed

Fuel consumption depends on other attributes like speed of the vehicle, time taken for the travel, type of roads, traffic, driving behavior, weather, weight, air conditioning, etc.

The problem - Attributes for fuel economy

Slide 7

Time to travel Speed Fuel consumed Distance travelled

Optimizing these objectives is known as Multi-objective optimization

The attributes/objectives for optimization in the example

Slide 8

A problem that takes more than one objective for optimization The objectives can be contradicting ones

Meaning, it is desirable to increase the value of one objective while decreasing the value of others

The objectives are represented as a vector and can be evaluated using The vector as a whole

Defining a evaluation function using the objectives Multi-objective optimization problem

Slide 9

In the current autonomous car agent problem the cost or evaluation function is taken as follows:

$\text{Cost} = \text{Time} + \text{Speed} + \text{Fuel} - \text{Distance}$

Time travelled and fuel consumed must have lower values for cost reduction.

Speed must be average (not less not more) Distance travelled must be maximum to reduce the cost

Cost function / Evaluation function

Slide 10

Choose the optimal attribute values to get minimum cost Goal = 0 cost Heuristic - Hill climbing

Choose attribute vectors at random (Stochastic hill climbing) Evaluate them

Retain the best while forgetting the less optimal values Heuristic

Slide 11

Performance - Safety, time, legal drive, comfort.

In the algorithm, reducing the cost of travel by optimizing the attributes taken

Environment - Roads, other cars, pedestrians, road signs.

Acceptable range of values for the attributes Time, Speed, Fuel and Distance travelled represented as vector of combination of these values

Actuators - Steering, accelerator, brake, signal, horn.

Initial solution, Candidate solution, Evaluation function.

Sensors - Fuel indicator, Speedometer, Digital clock in the display

Candidate solution with the acceptable range of values (from the sensors), number of iterations. PEAS

Slide 12

Step 1: Evaluate the initial state, if it is goal state then return success and Stop. Step

2: Loop Until a solution is found or there is no new operator left to apply. Step

3: Select and apply an operator to the current state. Step 4: Check new state Step 5: Exit.

Hill climbing algorithm - Steps

Slide 13

Code Walk through Stochastic hill climbing in PYTHON

Slide 14

```
import numpy as np import random; #The PEAS #Time, Speed, Fuel, Distance # Init_solution = [10, 100, 25, 10];
```

```
# Init_solution = [2,10,5,100] # Init_solution = [0, 0, 0, 0]; Iterations = 0 max_itr =1000 #
```

```
Can be increased or decreased Final_solution = None # Stores best of each iteration
```

```
#PEAS and the initial states
```

Slide 15

```
# Contains an objective vector of attributes to be optimized and the cost associated with the objectives
```

```
class Solu_space : def __init__(self): self.obj_vector = None self.cost = None def setObj_vector(self, val ):
```

```
self.obj_vector = val def setCost (self, cost): self.cost = cost def getAttr (self):
```

```
return {'Time, Speed, Fuel, Distance': self.obj_vector, 'Cost': self.cost } #Class : Solution space
```

Slide 16

```
# Function for calculating the cost - The evaluation function def calc_cost ( objVector ): cost = round((objVector [0] + objVector [1] + objVector [2] - objVector [3]),2) #print("Cost"+ str (cost)) return abs(cost)
```

```
#Function for cost evaluation
```

Slide 17

```
def gen_new_solution (): distance_step =0 speed_step = random.randint (10,100) time_step =round((random.randint (1,100))/60,2) try: distance_step = random.randint (1,round( speed_step *time_step,2)) except:
```

```
pass fuel_step = random.randint (1,25) objVector =[ time_step,speed_step,fuel_step,distance_step ]
```

```
#print("new solution: "+ str ( objVector )) return objVector #Function to generate new solution
```

Slide 18

```
def Hill_Climb ( objVector ): soln_space = Solu_space () soln_space.setObj_vector ( objVector ) oldCost =  
calc_cost ( objVector ); soln_space.setCost ( oldCost ) newobjVector = gen_new_solution () newcost =  
calc_cost ( newobjVector ) #Function Hill climb
```

Slide 19

```
if( newcost < oldCost ): soln_space.setObj_vector ( newobjVector ) soln_space.setCost ( newcost ) print(  
str  
( soln_space.getAttr ())) return soln_space.obj_vector print( str ( soln_space.getAttr ())) return  
soln_space.obj_vector # Function Hill climb - compare solution costs
```

Slide 20

```
def stopit (): globals () cost= calc_cost ( Final_solution ) if cost == 0: return True if Iterations ==  
max_itr : return True else: return False #Function - The stopping criterion
```

Slide 21

Calling all the defined functions

Slide 22

```
# Time,speed,fuel,distance Init_soln = [ random.randint (1,10), random.randint (1,100), random.randint  
(1,25), random.randint (1,500)] #Generate initial solution
```

Slide 23

```
Final_solution = Hill_Climb ( Init_solution ) #Initialize variable Final_solution print( Final_solution )  
while not stopit (): solution= Final_solution Final_solution = Hill_Climb (solution) print("Iteration no: "+  
str (Iterations)) Iterations+=1 print("Solution") print( str ( Final_solution [0])+ " Hrs " + str (   
Final_solution [1])+ " Km/Hr " + str ( Final_solution [2]) + " Litres " + str ( Final_solution [3]) + " Kms  
") print("Cost" + str ( calc_cost ( Final_solution ))) #Loop the Hill Climb
```

Slide 24

```
{'Time, Speed, Fuel, Distance': [1.3, 70, 12, 55], 'Cost': 28.3} Iteration no: 35  
{'Time, Speed, Fuel, Distance': [1.3, 70, 12, 55], 'Cost': 28.3} Iteration no: 36  
{'Time, Speed, Fuel, Distance': [1.3, 70, 12, 55], 'Cost': 28.3} Iteration no: 37  
{'Time, Speed, Fuel, Distance': [0.13, 10, 2, 0], 'Cost': 12.13} Iteration no: 172  
{'Time, Speed, Fuel, Distance': [0.13, 10, 2, 0], 'Cost': 12.13} Iteration no: 173
```

{'Time, Speed, Fuel, Distance': [0.13, 10, 2, 0], 'Cost': 12.13} Iteration no: 174
{'Time, Speed, Fuel, Distance': [0.13, 10, 2, 0], 'Cost': 12.13} Iteration no: 175
{'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 991
{'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 992
{'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 993
{'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 994
{'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 995
{'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 996 {'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 997
{'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 998
{'Time, Speed, Fuel, Distance': [1.5, 66, 2, 70], 'Cost': 0.5} Iteration no: 999 Solution 1.5 Hrs 66 Km/Hr 2 Litres 70 Kms Cost 0.5 Partial sample output

Slide 25

Add code to convert stochastic hill climb into Random restart hill climb

Solve the problem using Simulated annealing Implement the solution for the given problem using genetic algorithm That's all for Hill Climbing Food for thought

Topic-wise Collected Questions

`practice/sample_questions_topic_wise.md`

Repo-authored collected practice prompts.

Topic-wise Collected Practice Questions

Rendered from practice/sample_questions_topic_wise.md.

ACI Sample Questions Topic Wise

Collected from `Assign1\_PS7.pdf`, `Sample-questions-from-previous-year-exam.pptx`, and `ACI-LabSession2A-HillClimbing-TravelOptimization.pptx`.

Emergency Route Planning Using Greedy Best First Search

Source: `Assign1\_PS7.pdf`

1. Represent the city map as a graph.
2. Implement Greedy Best First Search in Python.
3. Use heuristic values to guide the search.
4. Print the visited nodes, optimal path, and total cost.
5. Compute the time complexity and space complexity from your implementation.
6. Explain the PEAS (Performance Measure, Environment, Actuators, Sensors) for the routing agent.
7. Explain the environmental characteristics for the given problem.
8. Explain the heuristic function used for the problem.

A* Search And Heuristic Evaluation

Source: `Sample-questions-from-previous-year-exam.pptx`

1. A Rescue Agent is deployed in a flood-affected city to deliver medicines from the base camp at `(X1, Y1)` to a relief shelter at `(X5, Y4)` on a `5 x 5` grid with allowed moves `Up`, `Down`, `Left`, and `Right`, where blocked and waterlogged cells cannot be entered.
2. Expand and depict the search tree for exactly 3 levels for the above problem.
3. Calculate the path cost and heuristic values for all nodes at Levels 1 to 3.
4. Apply A* search and show all steps.

Problem Formulation, PEAS, And Task Environment

Source: `Sample-questions-from-previous-year-exam.pptx`

1. A large supermarket chain wants to implement an Autonomous Checkout System where customers can pick items and walk out without scanning them manually.
2. Provide the complete problem formulation for the Autonomous Checkout System.
3. Provide the PEAS description for the Autonomous Checkout System.
4. Identify the various dimensions of the task environment with appropriate justification for each.

Hill Climbing Travel Optimization

Source: `ACI-LabSession2A-HillClimbing-TravelOptimization.pptx`

1. Choose the optimal attribute values to get minimum cost.
2. Evaluate the candidate solutions generated during hill climbing.

Sprint Mock 01

`practice/mocks/sprint-mock-01.md`

Repo-authored 30-mark mock practice paper.

Sprint Mock 01

Rendered from practice/mocks/sprint-mock-01.md.

Sprint Mock 01: AIMLCZG557 ACI

Time: 120 minutes

Marks: 30

Conditions: Closed book. Show search tables, path costs, heuristic values, and brief justifications.

Status: Original sprint mock based on local sample-question analysis, not an instructor paper.

Question 1: PEAS And Environment Classification [6 marks]

A hospital wants an autonomous medicine-delivery robot that moves through wards, avoids people, delivers medicine boxes, and returns to charging docks.

1. Write the PEAS description. [2]
2. Classify the task environment on observability, determinism, episodic versus sequential, static versus dynamic, discrete versus continuous, and single-agent versus multi-agent. Give one-line justification for each. [3]
3. State whether the agent should be rational or omniscient, and explain the difference. [1]

Question 2: Problem Formulation And Greedy Best-First Search [6 marks]

A rescue drone must travel from 'S' to 'G' using this graph:

S-A:2, S-B:4
A-C:2, A-D:5
B-D:1, B-E:4
C-G:5
D-G:3
E-G:2

Heuristic values to goal:

$h(S)=6$, $h(A)=4$, $h(B)=3$, $h(C)=4$, $h(D)=2$, $h(E)=1$, $h(G)=0$

1. Give the problem formulation: initial state, actions, transition model, goal

test, and path cost. [2]

2. Apply Greedy Best-First Search and list the expansion order. [2]
3. Give the path found and total path cost. [1]
4. State one weakness of greedy best-first search. [1]

Question 3: A* Search And Heuristic Quality [6 marks]

Use the same graph and heuristic from Question 2.

1. Apply A* search using $f(n)=g(n)+h(n)$. Show a table with node, g , h , and f for each expanded or frontier node. [3]
2. Give the final path and total cost. [1]
3. State the definition of admissibility. [1]
4. State the definition of consistency and explain why consistency is stronger for graph search. [1]

Question 4: Local Search, GA, And ACO [6 marks]

A timetable optimizer tries to minimize clashes, room changes, and late-evening classes.

1. Explain why this can be treated as a local search problem. [1]
2. Trace one hill-climbing iteration using this neighborhood:

Current score = 18 clashes
Neighbor 1 = 15 clashes
Neighbor 2 = 20 clashes
Neighbor 3 = 14 clashes

Choose the next state and justify. [1]

3. Explain local maxima, plateau, and ridge in this context. [1.5]
4. Describe the GA cycle for this problem: representation, fitness, selection, crossover, mutation. [1.5]
5. Give one high-level role of pheromone and heuristic desirability in ACO. [1]

Question 5: Game Playing, Alpha-Beta, And MCTS [6 marks]

Consider this left-to-right game tree. Root is MAX. Its children are MIN nodes.

Leaf utilities are:

Left MIN: 3, 5, 2

Middle MIN: 4, 6, 1
Right MIN: 7, 8, 9

1. Compute the minimax value of each MIN node and the root. [2]
2. Trace alpha-beta pruning left to right. State where pruning occurs, if any.

[2]

3. Explain why minimax assumes an optimal opponent. [0.75]
4. Describe the four MCTS phases and when MCTS is useful. [1.25]

Marking Discipline

After the timer ends:

- Search errors go to `./error-log.md` with the exact wrong open-list choice.
- PEAS/environment errors must be repaired with a new scenario.
- Alpha-beta errors must be repaired by retracing a smaller tree before retrying

this one.